Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

Published 03/28/2017
Copyright © 2017 SAE International
doi:10.4271/2017-01-1621
saepcelec.saejournals.org

Timing Analysis for Hypervisor-based I/O Virtualization in Safety-Related Automotive Systems

Andre Kohn and Karsten Schmidt
AUDI AG

Jochen Decker
Easycore GmbH

Maurice Sebastian
Symtavision GmbH

Alexander Züpke
RheinMain University of Applied Sciences

Andreas Herkersdorf
Technical University of Munich

ABSTRACT

The increasing complexity of automotive functions which are necessary for improved driving assistance systems and automated driving require a change of common vehicle architectures. This includes new concepts for E/E architectures such as a domain-oriented vehicle network based on powerful Domain Control Units (DCUs). These highly integrated controllers consolidate several applications on different safety levels on the same ECU. Hence, the functions depend on a strictly separated and isolated implementation to guarantee a correct behavior. This requires middleware layers which guarantee task isolation and Quality of Service (QoS) communication have to provide several new features, depending on the domain the corresponding control unit is used for. In a first step we identify requirements for a middleware in automotive DCUs. Our goal is to reuse legacy AUTOSAR based code in a multicore domain controller. In an example use case scenario, we analyze the end-to-end latency for transmitting and receiving CAN messages in a system using a hypervisor-based virtualization approach. A model-based timing analysis and an implementation on a state-of-the-art automotive microcontroller shall help to assess the usability for safety-related domains. Additionally, the timing behavior is compared to a common AUTOSAR implementation. We could show that a well-configured hypervisor and scheduling can provide similar results as an AUTOSAR implementation under certain circumstances. Furthermore, a model-based timing analysis allows evaluating different hypervisor configurations without physical hardware.

CITATION: Kohn, A., Schmidt, K., Decker, J., Sebastian, M. et al., "Timing Analysis for Hypervisor-based I/O Virtualization in Safety-Related Automotive Systems," *SAE Int. J. Passeng. Cars – Electron. Electr. Syst.* 10(2):2017, doi:10.4271/2017-01-1621.

INTRODUCTION

The increasing communication effort and data exchange of todays and future electronic vehicle components result in a considerably higher software complexity. This leads to a huge change in the automotive E/E architecture which is directly associated with numerous new challenges. The requirements for ECU architectures concerning real-time capability, CPU performance and memory management significantly differ, depending on the considered vehicle domain. However, isolation is one key aspect which is relevant for each domain and has to be considered in nearly all current and future automotive domain controller designs.

Generally, multicore CPUs provide more performance and are able to consolidate software of different Automotive Safety Integrity Levels (ASILs) on a single system. Another goal when applying highly integrated ECUs is to reuse legacy code of existing singlecore ECUs while minimizing the modification effort. This means, that different operating systems might be run on the same controller. All these use cases imply the need for an efficient isolation concept to avoid mutual interferences.

The controllability of highly-integrated multicore domain controllers can be improved by a well-conceived middleware that manages the access to resources and peripherals. An established isolation approach in the IT domain is a hypervisor-based system-virtualization. In

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

contrast to embedded microcontrollers, the IT domain uses microprocessors which provide hardware-support for virtualization. Due to the fact that most hardware architectures for safety-related automotive electronic systems, do not provide such a hardware support, only software-based virtualization approaches can be considered. Previous implementations are mostly based on para-virtualization because the hardware architectures do not fulfill the requirements for a full-virtualization of the Popek & Goldberg theorem [1]. Even though virtualization provides several advantages such as a strong isolation of virtual machines (VMs), it is still very uncommon in automotive ECUs. This is often due to the disadvantageous overhead in timing and memory resources

In our contribution, we analyze the timing behavior respectively the timing overhead of the ecHypervisor by Easycore GmbH. The evaluation target hardware is a NXP/Freescale MPC5748G CPU using the PowerPC architecture. Our goal is to consolidate tasks from AUTOSAR-based ECUs from the chassis domain which communicate with each other but also with an ECU from the motor domain. Here, more than one AUTOSAR system runs in parallel on the target hardware

In the second section we present similar approaches and evaluations of related publications. Section 3 gives an overview about requirements for system design and the architecture of target platform. The implementation and applied hardware and software architecture is described in Section 4 before we discuss our evaluation results and conclude in Sections 5 and 6.

RELATED WORK

System virtualization is well established in different software domains to isolate and separate several applications. Desktop and server-based systems often use hardware support of the processor architecture such as Intel VT-x and AMD-V which allow implementing full-virtualization and hence do not require a modification of the guest systems. A similar approach for embedded systems was presented by ARM with the introduction of the ARMv7 architecture. Controllers which use this architecture can often be found in smartphones and tablet PCs. Another field of application for these CPUs are automotive infotainment systems which benefit from the increased performance and the ability to isolate different operating systems. However, virtualization is a new field of application for automotive electronics so that implementations are rare.

Generally, most available hypervisor solutions for embedded systems use a microkernel. Popular approaches are the Sysgo PikeOS, GreenHills Integrity, WindRiver Hypervisor and the L4-based OKL4.

The features of PikeOS were evaluated by Continental AG who built an In-Vehicle Infotainment (IVI) demonstrator which combines an AUTOSAR system with a POSIX-based security stack. Additionally, the Continental-specific Open Infotainment Platform (OIP) which uses GENIVI and a partition running Android with custom-oriented apps were implemented. The operating systems either use Asynchronous Multi-Processing (AMP) with several partitions on one core or Synchronous Multi-Processing (SMP) with only one partition on the same core. The implementation runs on an ARM Cortex-A9 processor while a port for the Cortex-A15 providing hardware support for virtualization is still under development. This demonstrator shows a possible software architecture with system virtualization for the infotainment domain [2].

In the context of the ARAMiS project, an automotive demonstrator was developed to evaluate the RTA-HV, a hypervisor by Etas Ltd., for the applicability in highly-integrated automotive ECUs. The demonstrator uses the Infineon TriCore architecture, more precisely the AURIX TC277, running two individual automotive software systems with AUTOSAR Basic Software and AUTOSAR-OS each. The first system consisted of the BMW AUTOSAR Core (BAC) with different applications, while the second system runs a powertrain application [5].

A continuative work was the development of a cost model to determine the workload, resource utilization and timing behavior of two implementations. The first one is based on an intrusive approach with an AUTOSAR-only system, while the second, non-intrusive implementation applies the RTA-HV. A disadvantage of the hypervisor was that only one partition can be embedded on the same core [4].

Virtualization of a CAN controller and a mechanism called pre-virtualization were realized on the Infineon AURIX TC27x in another contribution. Pre-virtualization combines the advantages of a full-virtualization approach with the efficiency of a para-virtualization approach. The efficiency improvement is achieved by applying binary translation before the final implementation. The comparison of the clock cycles and execution time of the virtualized instructions show the efficiency of this approach. Furthermore, the work lists general requirements for a hypervisor from the automotive point of view [3].

An AUTOSAR-compatible microkernel was presented by Elektrobit Automotive GmbH. The authors came to the conclusion that the kernel performance is comparable to that of a standard operating system [6].

REQUIREMENTS OF AUTOMOTIVE SOFTWARE ARCHITECTURES

Functional safety is the most relevant aspect for domains with classic automotive applications, e.g. chassis and powertrain. Hence, highly-integrated ECUs have to comply with the requirements of the ISO 26262 standard, namely freedom-from-interference and hard real-time constraints. Furthermore, aspects such as static memory allocation and scheduling have to be considered. This means, distribution of memory must be applied in the design phase, while during execution, memory accesses are protected by a Memory Protection Unit (MPU). In the last years, AUTOSAR has become the standard for software architectures in such safety-related domains due to its static implementation approach. Hence, only hardware architectures with an MPU are supported which means dynamic

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

memory allocation is not possible. Although, gateway ECUs mostly do not necessarily execute applications with hard real-time requirements, AUTOSAR is used in these control units as well.

Automotive Middleware

An ECU software architecture consists of an application layer, a middleware and the basic software. The middleware is a more generic designation of a runtime environment and coordinates the complete communication and data transfer between applications and also the access to peripherals. This includes sensors, actuators and memory registers. Depending on the domain, the middleware can be configured and adapted to the user's needs while in the automotive domain AUTOSAR RTE is a well-established standard.

Furthermore, the middleware usually offers services and capabilities to virtualize shared hardware and software resources. This allows to encapsulate functions to avoid interferences and to port legacy code from a singlecore to a multicore system. Especially applications running on other operating systems such as Linux or Android often have to be isolated from the rest of the system.

From the application software perspective, the middleware must be able to trigger the application software respectively the user functions. The segregation in space and time but also the management of memory and communication must be handled accordingly. This implies communication between applications on the same ECU and with peripheral to realize a data exchange with other ECUs. Furthermore, an inter-core communication mechanism has to be implemented in multicore CPUs. In case of using more than one operating system on the same CPU, virtualization is a promising approach. Further non-functional requirements for a middleware on nearly all automotive multicore ECUs are the following:

- Guarantee of real-time and performance requirements
- Realization of a safe and secure system
- Handling of multiple operating systems / multi-core capable OS
- Handling applications of different ASILs
- Management of shared resources
- Support of fail-operational
- Support for legacy code applications

In general, hypervisor solutions are less popular for AUTOSAR-based real-time domains and there are only few approaches already available. Furthermore, the approaches are limited and optimized to a specific architecture while a port to another architecture is a cost-intensive task. The virtualized system is often limited by the fact that the hypervisor does not allow implementing more than one partition on the same core. Multiple partitions on one core require an additional effort for temporal separation because each guest system applies its own scheduling policy which has to be synchronized to the other guests. So, scheduling and memory management are key aspects in the system design. Thus, our goal is to use more than one partition on one core while applications functionality must not be influenced.

Temporal and Spatial Separation in Multicore Systems

The implementation of several partitions on a singlecore CPU or on a dedicated core of a multicore system requires an exact and sophisticated system design concerning timing and memory allocation.

Generally, for system virtualization in a multicore system, a partitioned scheduling is an appropriate method to statically allocate tasks during system design. Each guest system respectively virtual machine uses its own operating system and scheduling algorithm. Thus, a virtual machine consists of a taskset and the corresponding scheduling policy which is defined by the guest operating system. If the guests have different requirements for functional safety, meaning different ASILs, the corresponding access rights and priorities have to be allocated to the virtual machines. Hence, virtual machines have different quality of services and can be activated, deactivated and rebooted by the hypervisor. For virtual machine scheduling implemented by the hypervisor, each VM needs a minimum of CPU resources to run.

Due to the fact that virtual machines are handled as operating system tasks, a second scheduling layer is necessary. Hence, a two-level scheduling scheme is applied. The first level is part of the hypervisor and schedules virtual machines while the second level is used within the virtual machines. For the former, a static time partitioning can be used so that each virtual machine has its own time slot for operating. Depending on the priority, the time slot length can be individually defined. An example for an implementation with two VMs, each running two tasks, is illustrated in Figure 1.

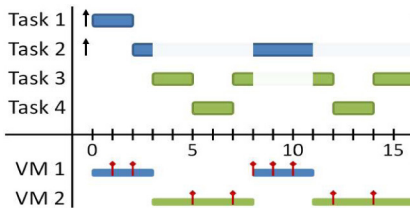

Figure 1. Example for hierarchical scheduling with time partitioning

Generally, schedule design is a complex and time-consuming challenge. An optimal VM scheduler is exactly synchronized with the run-time of the guest system tasks and the task cycle.

As an alternative approach to time partitioning, the required bandwidths of a task and a virtual machine can be calculated. This approach can be adapted to the bandwidth requirements of a virtual machine with its cyclic taskset and scheduling algorithm. With the help of the appropriate function, the required resources of the VM can be derived [8]. Furthermore, a periodic resource model helps to allocate specific timing budgets of a resource so that the maximum resource consumption can be determined. A prerequisite of such a model is that guest system tasks still can meet the defined real-time requirements. This means, the requested CPU time of a virtual machine must always be same or equal as the cumulative CPU time which can be used of the VM [9]. Hence, a mapping of several virtual machines to cores is only possible in a powerful system which provides a suitable VM scheduling.

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

Generally, the main challenges of virtualization in multicore systems are the partitioning and the allocation of virtual machines to cores and the hierarchical scheduling on separate cores.

In addition to temporal separation, an important aspect for a static system is the independency of virtual machines. Shared resources should be limited to CPU and communication bus accesses. Thus, only static and exclusive allocation of memory should be taken into account which is hard to be realized by a memory management unit (MMU) using a dynamic page table. However, an MMU can be configured to use static page tables as well but unfortunately, there is no MMU-based controller available which is certified to execute functions which require an ASIL-D.

Hence, spatial separation respectively memory management in safety-related vehicle ECUs with ASIL-D functions can only be realized by an MPU. The memory is statically partitioned and allocated by the linker at compile time so that during run-time no changes have to be made. Moreover, an MPU is much cheaper to implement which results in reduced costs for a complete ECU.

APPROACHES FOR VIRTUALIZATION IN EMBEDDED AUTOMOTIVE SYSTEMS

The microcontrollers used in automotive ECUs mostly do not allow a full-virtualization approach due to sensitive instructions which can be called from the user mode instead of the full-blown supervisor mode. The unmodified operating systems within the virtual machines require the supervisor mode for hardware access and hence a superior hypervisor mode is necessary to run several independent operating systems on the same hardware in parallel. This mode is implemented by hardware-support for virtualization such as Intel-VT-x and AMD-V or in the ARM Cortex-A family. In contrast, automotive microcontrollers do not provide such a virtualization support so that the guest systems have to be modified accordingly.

Para-virtualization

Today, para-virtualization is often the only approach to benefit of virtualization in embedded microcontrollers while commercial products are rare. Furthermore, operating systems which require an MMU cannot be executed in a virtual machine implemented on safety-related controllers with an MPU. The execution of an AUTOSAR partition in parallel with an Android system is hence not possible. Instead, several AUTOSAR guest systems, each with a different revision, are considerable. In a para-virtualized system, critical instructions are replaced by hypercalls which directly call the hypervisor. In contrast to trap-and-emulate virtualization, para-virtualization reduces run-time overhead by omitting the costly trapping and code checking at run-time.

In a virtualized multicore system, either a single virtual machine or more than one VM could be executed on the same core. The latter case requires a second scheduling level in the hypervisor and as an addition a core scheduler which manages access to the virtualized resources. The effort to synchronize both schedulers is a huge challenge while the complete system becomes much more complex.

Microkernel

Generally, a hardware architecture provides a privileged and an unprivileged mode while the kernel, as a central component of most operating systems, runs in the privileged mode. Hence, it has full access to the hardware, while applications execute unprivileged and have limited access rights. Common desktop OSes typically use a monolithic kernel where all OS services such as memory management, process management, drivers and the file system execute in the privileged supervisor mode. This results in a big Trusted Code Base (TCB) and a fault in a driver could crash the whole system.

In contrast, microkernels use only a minimal privileged software layer which provides general methods concerning OS services and user applications. They provide address spaces as containers similar to page tables to control memory accesses. Furthermore, microkernels use scheduler activations or threads to abstract execution time. And lastly, microkernels provide means for communication between executing entities

In a microkernel implementation, the software resources are separated into kernel mode or user mode. Generally, a microkernel layer can be used as a hypervisor to prevent the direct access to the hardware resources. The software processes are encapsulated in their own hardware address while the communication with other parts of the system is only possible by invoking kernel mechanisms.

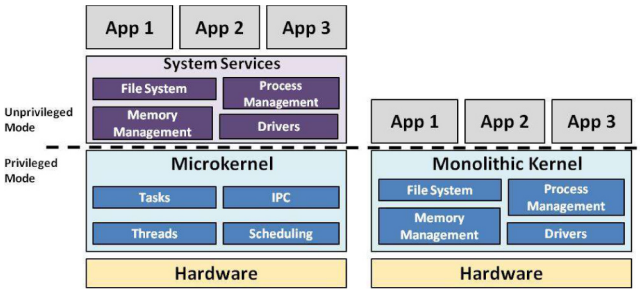

Figure 2. General structure of a microkernel and monolithic kernel

A hypervisor can either be based on a microkernel or on a monolithic kernel. The latter implements memory management mechanisms, scheduling as well as the device drivers and hence has a big TCB which is ineffective in real-time systems. Instead, a microkernel-based hypervisor is much smaller and implements only the basic functionality while device drivers a moved to the partitions. An advantage is that the drivers are only used in those partitions which uses the corresponding resource. Furthermore, the hypervisor layer is minimized which leads to a small TCB. The main architecture differences of a monolithic and microkernel can be found in Figure 2.

AUTOSAR Communication Stack

Generally, AUTOSAR provides the COM basic software module to realize bus communication. Each applied bus technology uses the same protocol stack structure while the AUTOSAR signals are consolidated to signal groups. One or more signal groups form an Interaction Layer Protocol Data Unit (I-PDU) which can be handled by the COM module. The I-PDU length depends on the maximum

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

Data Link Layer Protocol Unit (L-PDU) length of the applied communication interface. The COM module routes the I-PDUs to the PDU router before it is sent to the communication interface.

A sending request for a CAN message is initiated by the functions *Com_TriggerTransmit()* and *CanIf_Transmit()* of the COM and PDU Router. In a next step, the CAN interface writes the message to the sending buffer of the CAN controller by calling the function *Can_Write()* function. A successful transmission to the bus is indicated by a *CanIf_TxConfirmation()*call. The feedback is then forwarded to the upper layers by applying *PduR_TxConfirmation()* and *Com_TxConfirmation()*.

An incoming CAN message is indicated by a hardware interrupt in the CAN controller. The CAN driver informs the CAN interface about this message by calling *CanIf_RxIndication()*. Finally, this information is fowarded to the other layers by the functions *PduR_RxIndication()* and *Com_RxIndication()*. An overview about the sending and receiving mechanism for CAN messages in AUTOSAR is illustrated in Figure 3.

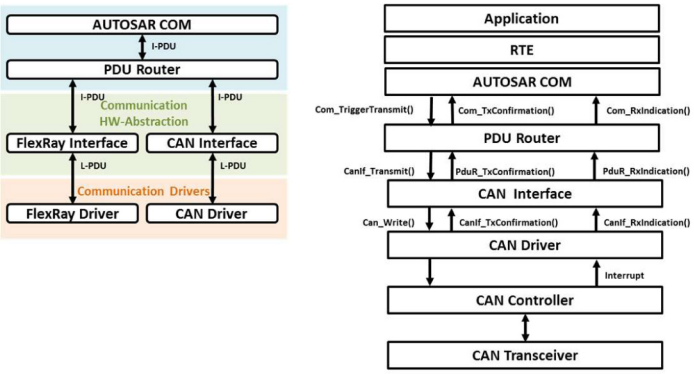

Figure 3. Sending and receiving CAN messages in AUTOSAR [7]

IMPLEMENTATION

The implementation of a real world use case requires an analysis of the E/E architecture of a close-to-production vehicle. An approach for future vehicles is to realize the communication between the domains with the help of multicore DCUs and a central gateway. Furthermore, a consolidation of different domains in high performance control units is considerable on the long run. Especially chassis and motor functions in electric vehicles often have to be considered as a combined function for longitudinal or lateral acceleration with mutual influence. This means, braking and acceleration can be handled by an electric motor as well. If legacy code from both ECUs has to be implemented on a DCU, the communication between them has to be virtualized.

The goal of our contribution is to evaluate the ecHypervisor timing overhead, especially when sending CAN messages. In our implementation, we analyze the communication between two ECUs from the chassis domain and the motor domain. The ECUs from both domains run parts from common functions (e.g. start/stop) and hence exchange according CAN messages. Thus, a consolidation of ECUs requires CAN virtualization and an isolation of the ECU software in partitions. The DCU has to route incoming messages to the appropriate partition and to send messages from one partition to another via a virtual CAN bus or to the real CAN bus.

The goal of our work is to evaluate the overhead in time respectively the end-to-end latency as a result of the hypervisor implementation. The chosen CAN messages are based on real messages which are exchanged between an Electronic Power Steering (EPS), an Electronic Stability Control (ESC), a motor ECU and gear ECU. The periodic messages use a cycle time between 5 ms and 200 ms and are listed in Table 1.

Table 1. Examples for exchanged CAN messages between chassis and motor domain

	Sender	Receiver	Cycle Time / ms
ESP_01	ESC	Motor, EPS	20
ESP_02	ESC	Gear	20
ESP_03	ESC	Motor, EPS	200
ESP_04	ESC	Motor, Gear, EPS	10
ESP_05	ESC	Motor, EPS	20
ESP_06	ESC	Motor	5
Chassis_01	Damper	ESC, EPS, Gateway	25

Most of the messages are send from the ESC-ECU either to the EPS-ECU or to the motor domain. The sender *Damper* represents another ECU of the chassis domain which is not considered in this implementation.

Virtualization and Communication

For the evaluation we consolidate the ESC and the EPS-ECU in one Chassis ECU. We use one so called resource partition for each ECU (vECU1 and vECU2) while a third resource partition includes the required CAN driver. The communication between the virtual ECUs and the driver partition is realized by the virtual modules *vCan* and *vCanIf* which use the same interfaces as the original AUTOSAR modules. Hence, a modification of the AUTOSAR basic software and the CAN driver is not necessary. Furthermore, the *vCanIf* module allows a message exchange between vECU1 and vECU2 without communicating over the real CAN bus. The virtual ECUs are configured to individually use a defined amount of message buffers.

When receiving a CAN message from the real bus, the hypervisor routes it from the CAN driver in the driver partition. The virtual CAN interface sends the message to the virtual CAN module of the receiving partition. Here, the function *CanIf_RxIndication()* is called (Figure 4, left).

Sending from a virtual ECU, e.g. vECU1, to the real bus is initiated by calling *CanIf_Transmit()* on the virtual CAN module. The message is then written to a shared memory of the virtual ECU. In a next step, the driver reads this data from the shared memory and routes it to the real bus. A successful transmission to the bus is signalized by a function call of *CanIf_TxConfirmation()* in the virtual ECU (Figure 4, right).

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

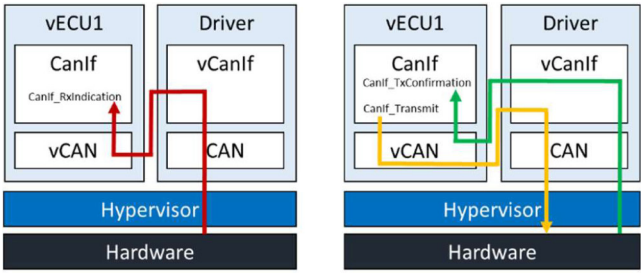

Figure 4. Communication paths when receiving (left) and sending (right) CAN messages between bus and virtual ECU

If an incoming message has to be routed to more than one virtual ECU, the hypervisor first sends it to the CAN driver which identifies the receivers with the help of a routing table. The data is then written to the shared memory of the virtual ECUs and the vCAN uses *CanIf_RxIndication()* to signalize the availability of the new message (Figure 5, left).

Figure 5 (right) shows the approach for sending a CAN message from vECU1 to vECU2 and also to the bus. The function *CanIf_Transmit()* of the virtual CAN module in vECU1 is called and the message is written to the shared memory. The vECU1 then informs the driver partition before the driver identifies the receiver. In this example, the driver reads the data from the vECU1 memory and transfers it to the bus. In parallel, the message is copied to the memory of vECU2 which is informed about the new available data.

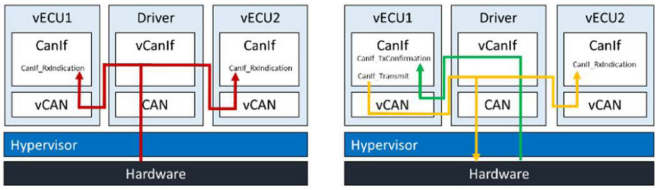

Figure 5. Communication paths when receiving a CAN message for both vECUs (left) and sending from vECU1 to vECU2 and the physical bus (right)

Task Scheduling and Time Partitioning

The task scheduling is based on the timing requirements of the CAN messages in Table 2. All ESP_xx messages are allocated to the cyclic 5 ms task *ComTask.* This period defines the shortest cycle time of the considered messages. The *ComTask* is similar to the function *Com_MainFunctionTx()* of the AUTOSAR standard and is executed in parallel to the *AppTask* on vECU1. Additionally, vECU1 and vECU2 run two event-triggered tasks for sending and receiving interrupts *(CanIsrTx* and *CanIsrRx).* In our implementation, the task priority is in an ascending order. The configuration of the task scheduling is listed in Table 2.

Table 2. Configuration of task scheduling

Task	Partition	Priority	Cycle Time / ms
CanIfMainFunction	Driver	17	event-triggered
CanIsrRx	vECU2	10	event-triggered
CanIsrTx	vECU2	10	event-triggered
CanIsrRx	vECU1	15	event-triggered
CanIsrTx	vECU1	30	event-triggered
AppTask	vECU1	200	5
ComTask	vECU1	20	5

Furthermore, different scenarios for time partitioning are defined which allows to statically allocate resources to the applications. In our configuration, a cycle for the time partitioning schedule is 5 ms to meet the requirements of the ESP_06 message period. We apply an exclusive allocation which means that each resource partition has its own time partition. Moreover, we analyze four different approaches to schedule the driver partition.

The first and second approach imply a separate time partition for the driver so that each resource partition has a guaranteed run-time and resource access. The driver partition is either executed after both resource partitions (Scenario 1) or after each resource partition (Scenario 2). In the latter case, the vECUs can send and receive messages directly after running. However, a disadvantage of this scenario is that four partition switches in one time partition cycle are required. This is due to the fact that the driver partition is executed twice.

Furthermore, run-time for the resource partitions is shortened from 2 ms to 1.5 ms. The amount of partition switches can be reduced from four to three if Scenario 1 is applied. Both scenarios are illustrated in Figure 6.

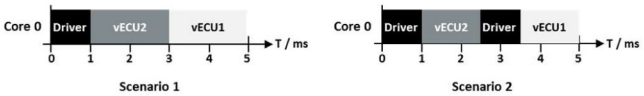

Figure 6. Approaches for time partitioning with separate driver partition

In the third scenario, the driver and one vECU share a time partition. This means, each remaining time partition has a timing budget of 2.5 ms. This approach is applicable if the applications in the shared time partition do not require guaranteed uninterrupted execution time. Generally, within the partition, the task with the highest priority is executed first. Due to the fact that hardware interrupts always have the highest priority, the currently running OS tasks in this partition could be interrupted by the CAN controller (Figure 7, left).

The fourth scenario is nearly identical to Scenario 3. The main difference is that two CPUs cores are used instead of a single core. Hence, Core 0 runs the shared time partition while Core 1 executes the second time partition with vECU2 (Figure 7, right).

Furthermore, we compare the third and the fourth scenario with a common implementation of AUTOSAR.

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

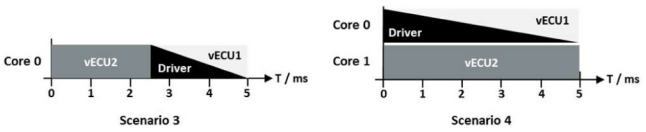

Figure 7. Approaches for time partitioning with integrated driver on singlecore (left) and multicore (right)

Hardware Architecture

For our implementation, we use an evaluation board with a NXP/Freescale MPC5748G controller. The hardware architecture is based on the 32-bit PowerPC architecture and uses two e200z4 cores in parallel with one additional e200z2 core. The two e200z4 cores are clocked with 160 MHz each and provide 8 KB instruction cache and 4 KB data cache each. Furthermore, the z4 cores provide a Floating Point Unit (FPU) and use Variable Length Encoding (VLE) which significantly improves the code density. In contrast, the e200z2 core runs at a clock rate of 80 MHz and cannot access its own cache memory. A crossbar switching architecture is used to access the shared memory consisting of 6 MB EEPROM and three blocks of 256 KB SRAM. The crossbar uses a System Memory Protection Unit (SMPU) providing 32 access windows and a granularity of 16 bytes to protect the system from unauthorized memory accesses. Additionally, different system bus masters access the peripheral by using the crossbar and a peripheral bridge. Error Correcting Coding (ECC) is available for cores, crossbar, peripheral bridge and shared memory to correct data transfer errors. An overview of the applied MPC5748G architecture is illustrated in Figure 8.

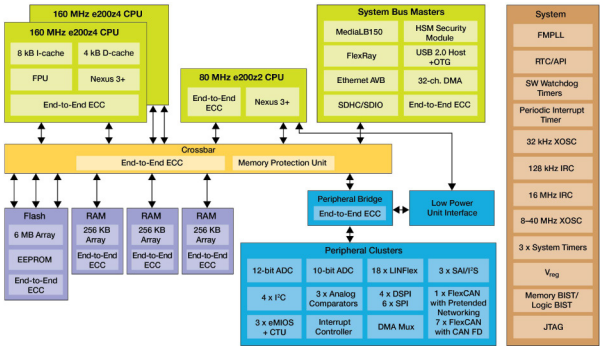

Figure 8. Block diagram of the NXP MPC5748G architecture [10]

Definition of Measuring Points

The analysis of the hypervisor end-to-end latency for the CAN communication of virtualized ECUs requires the definition of specific measurement points. The time for receiving a CAN message could be determined by setting the starting point at the entry point of the hypervisor interrupt handler. The timer is stopped when the function *CanIf_RxIndication()* of vECU1 is called (Figure 4).

Furthermore, the sending time from vECU1 to the real bus is measured with the help of measurement points at the function call *CanIf_Transmit()* and when writing to the registers of the CAN driver.

The third timing measurement considers message transmission from vECU1 to vECU2. In this case, the time from the function call *CanIf_Transmit()* in vECU1 until the call of *CanIf_RxIndication()* in vECU2 is determined (Figure 5, right). Finally, for the transmission to the bus, the path from *CanIf_Transmit()* in vECU1 until *CanIf_RxIndication()* in vECU1 is required.

Model-based Timing Analysis

The instrumented reference implementation of the hypervisor allowed to measure the execution times with respect to a virtual driver *(vCan),* a virtual interface *(vCanIf* and the physical driver *(CAN).* Due to the optimal geared schedules and the missing ECU/COM requirements, timing effects of the scheduling could not be identified. For this, we used a virtual integration model to simulate the timing behavior for the sending and receiving processes. Our focus was on the singlecore scenarios 1, 2 and 3, while the fourth scenario was not considered because of the missing time partitioning. The different scenarios and configurations were analyzed with the help of SymTA/S by Symtavision GmbH. This tool allows to model the software architecture and the corresponding timing.

The sending of the message ESP_01 starts by calling the functions *Tx_COM_EcuESP_ESP_1* and *vCAN_write_ESP_1* within the *ComLayer_Tx* task of the source partition vECU1. Generally, the task has a 5 ms cycle while the runnable *Tx_COM_EcuESP_ESP_1* has a cycle time of 20 ms. Hence, this runnable is only executed every fourth task cycle. In the next step, the runnables *vCanIf_Transmit* and *Can_Write_ESP1* are called which are both implemented in the *CanIf_MainFunction* of the CAN driver partition. This process is also applied to the messages ESP_02 to ESP_06.

The receiving process from the Fahrwerk_01 ECU is started by an interrupt indication of the hypervisor interrupt handler *(Handler_Fahrwerk_1).* The message is then forwarded to the driver partition *(Rx_CAN_Fahrwerk_1)* before it is routed to the destination partition vECU1 (ESC-ECU). The corresponding paths are exemplary illustrated in Figure 9.

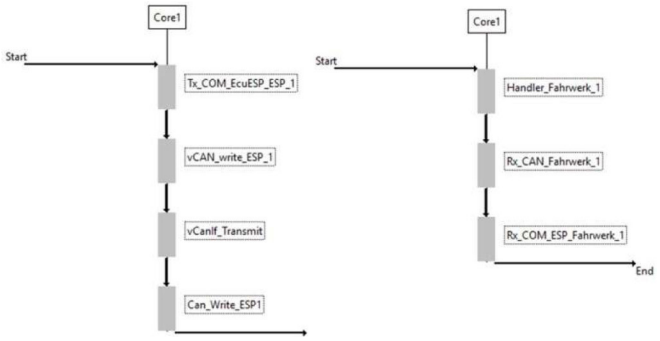

Figure 9. Paths from model-based timing analysis for sending (left) and receiving (right) CAN messages

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

Results

For each of these paths, 1000 measurements were applied to determine the minimal, the maximum and the average time.

The sending process shows the influence of the scheduling on the end-to-end latency. In the first and the second scenario, the message is sent at the beginning of the time partition. Hence, the time partition of the source has to be finished before the driver partition is active to forward the message to the bus or the destination partition. The worst case latencies of these scenarios are about 2 ms (scenario 1) and 1 ms (scenario 2). In contrast, the sending latency for scenario 3 is only 0.043 ms while we measured 0.113 ms for scenario 4. An overview can be found in Figure 10 (see Appendix).

The third scenario consolidates the CAN driver partition and the vECU1 in the same time partition. Hence, this approach is very close to a common AUTOSAR implementation. A comparison of these implementation shows a similar sending latency even though a time partitioning is applied in scenario 3. In some cases, the virtualization is even better which is due to the fact that in an AUTOSAR system, the sending process can be interrupted by an incoming message. The comparison is illustrated in Figure 11 (see Appendix).

In contrast, the worst case end-to-end latencies for the receiving process are much higher, especially for scenarios 1, 2 and 3. We measured 6.882 ms until the message arrived in vECU1 and 4.855 ms for vECU2. The reason for this are the waiting times until the driver partition is executed and the interrupt can be handled. Furthermore, the CAN bus does not guarantee a deterministic data transmission so that the time partitioning can be asynchronous to the bus. The minimal latency was measured in the AUTOSAR implementation (0.026 ms). The results can be found in Figure 12 in the Appendix.

For a direct comparison with AUTOSAR we chose scenario 4 which uses a consolidated driver partition and application partition. The measured overhead is very similar while the timing overhead of the hypervisor implementation is minimal (0.014 ms). The comparison is also illustrated in in Appendix (Figure 13).

In parallel to the measurements of the physical ECU, the modelbased timing analysis helped a lot to identify the reasons to the latencies, especially for the receiving process.

As an example we assume the ESC as the destination partition in scenario 1 (cp. Figure 6). The best case latency is measured when the interrupt arrives at the end of the driver partition execution. The system has to wait about 2 ms until the message arrives in the ESC ECU (Figure 14). In the worst case, the interrupt arrives at the beginning of the EPS partition so that we have to wait until the next execution of the driver partition where the interrupt is handled. However, the EPS partition is executed before the ESC partition which consumes additional time. Finally, a worst case latency about 7 ms can be expected (Figure 15).

In scenario 2 the driver partition is called twice per time partition cycle. Again, the latency is minimal when the interrupt arrives at the end of the driver partition. In the next step, the message can be directly forwarded to the destination partition (Figure 16). The worst case latency the interrupt of the incoming CAN message is at the beginning of the ESC partition execution. After 2 ms it can be handled by the driver partition before the EPS ECU time partition is finished. The result is a worst case latency of about 5 ms (Figure 17). Finally, in scenario 3 the driver partition is implemented in the same time partition as the ESC ECU. So, when a CAN message for the ESC arrives during the execution of this partition, the latency is minimal. In the worst case, the interrupt has to wait 2.5 ms until the execution of the EPS time partition is completed (Figure 19).

SUMMARY/CONCLUSIONS

The timing overhead of the I/O virtualization strongly depends on the implementation scenario and the corresponding configuration. This means, the communication need of the application partitions have to be considered during the software design of the ECU so that the order of the time partitions can be derived.

The configuration of a hypervisor, the scheduler and the time partitioning definitely requires deep knowledge about the timing requirements of the ECU functions. If legacy code should be reused, it is not possible to just put it into a partition even though it is modified for para-virtualization.

When using a time partitioning approach, the requirements of the applications have to be considered. If a low latency is needed, an implementation close to AUTOSAR is recommended, e.g. with no separate driver partition. In contrast, if a guaranteed worst case execution due to ASIL requirements is essential, a separation of application and driver partition is considerable. In this case, an incoming hardware interrupt is not processed during an active application partition.

On the one hand, we measured the end-to-end-latencies for sending and receiving CAN messages on a physical ECU while on the other hand, we used a model-based timing analysis for evaluation. The delta for the receiving scenarios is very low and between 0.001 ms and 0.024 ms while we measured a delta of 0.426 ms to 1.449 ms for the sending scenario. Generally, it has to be considered that we have 1000 measurements compared to 100 traces, so we assume that the more traces are used, the smaller the delta is.

As a conclusion we can say, that the tool-based timing analysis is very helpful to evaluate a hypervisor without a physical ECU hardware. It allows to efficiently test different scenarios without a time-intensive implementation.

In a subsequent work we will also evaluate the spatial overhead of a hypervisor-based I/O virtualization. Furthermore, other buses such as FlexRay and Ethernet have to be taken into account, especially for DCUs in domain-oriented E/E architectures.

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

REFERENCES

1. Popek, Gerald J., and Goldberg Robert P. "Formal requirements for virtualizable third generation architectures." *Communications of the ACM* 17.7 (1974): 412–421.
2. Posch, T., "Hypervisor separates software worlds in the dashboard". In: EETimes Europe (2014).
3. Schneider, R., Kohn, A.,Schmidt, K., Schoenberg, S., "Efficient Virtualization for Functional Integration on Modern Microcontrollers in Safety-Relevant Domains," SAE Technical Paper 2014-01-0206, 2014, doi:10.4271/2014-01-0206.
4. Juergens, D.,Reinhardt, D., Schneider, R., Hofstetter, G., "Implementing Mixed Criticality Software Integration on Multicore -A Cost Model and the Lessons Learned," SAE Technical Paper 2015-01-0266, 2015, doi:10.4271/2015-01-0266.
5. Reinhardt, D. and Morgan, G., "An embedded hypervisor for safety-relevant automotive E/E-systems". In: Proceedings of the 9th IEEE International Symposium on Industrial Embedded Systems (SIES 2014) (June 2014), pp. 189–198. doi :10.1109/SIES.2014.6871203.
6. Haworth, D. "An AUTOSAR-compatible microkernel for systems with safety-relevant components," Informatik aktuell, vol. Herausforderungen durch Echtzeitbetrieb, pp. 11–20, 2012.
7. Zimmermann, W. and Schmidgall, R., "Bussysteme in der Fahrzeugtechnik", Springer Fachmedien, 2006
8. Groesbrink, S. and Almeida, L., "A criticality-aware mapping of real-time virtual machines to multi-core processors." Emerging Technology and Factory Automation (ETFA), 2014 IEEE. IEEE, 2014.
9. Shin, I. and Lee, I., "Compositional real-time scheduling framework with periodic model." ACM Transactions on Embedded Computing Systems (TECS) 7.3 (2008): 30 .
10. NXP Semiconductors, "MPC574xB/C/D/G Series Microcontrollers REV 3", Product Sheet, NXP, 2016

CONTACT INFORMATION

Andre Kohn, M.Sc.
AUDI AG
85045 Ingolstadt, Germany
andre.kohn@audi.de

Dr.-Ing. Karsten Schmidt
AUDI AG
85045 Ingolstadt, Germany
karsten.schmidt@audi.de

Dipl.-Ing. (Univ.) Jochen Decker
easycore GmbH
91508 Erlangen, Germany
jochen.decker@easycore.de

Maurice Sebastian
Luxoft/Symtavision GmbH
38122 Braunschweig, Germany
msebastian@luxoft.com

Dipl-Ing. (FH) Alexander Züpke
RheinMain University of Applied Sciences
65195 Wiesbaden, Germany
alexander.zuepke@hs-rm.de

Prof. Dr. sc. techn. Andreas Herkersdorf
Institute for Integrated Systems
Technical University of Munich
80290 München, Germany
herkersdorf@tum.de

DEFINITIONS/ABBREVIATIONS

AMP - Asynchronous Multi Processing

ASIL - Automotive Safety Integrity Level

BAC - BMW AUTOSAR Core

CAN - Controller Area Network

CanIf - CAN Interface

CPU - Central Processing Unit

DCU - Domain Control Unit

ECC - Error Correcting Code

ECU - Electronic Control Unit

EPS - Electronic Power Steering

ESC - Electronic Stability Control

FPU - Floating Point Unit

HV - Hypervisor

ISR - Interrupt Service Routine

IVI - In-Vehicle Infotainment

MPU - Memory Protection Unit

MMU - Memory Management Unit

OIP - Open Infotainment Platform

OS - Operating System

PDU - Protocol Data Unit

RTE - Runtime Environment

RX - Receive

SMP - Synchronous Multi Processing

SMPU - System Memory Protection Unit

TCB - Trusted Code Base

TX - Transmit

vCAN - Virtual CAN

vCanIf - Virtual CAN Interface

vECU - Virtual ECU

VLE - Variable Length Encoding

VM - Virtual Machine

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

APPENDIX

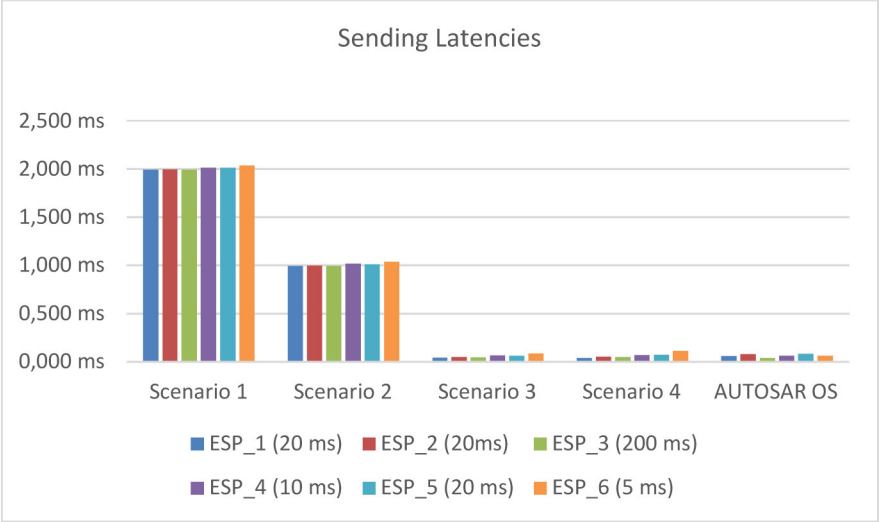

Figure 10. Sending latencies

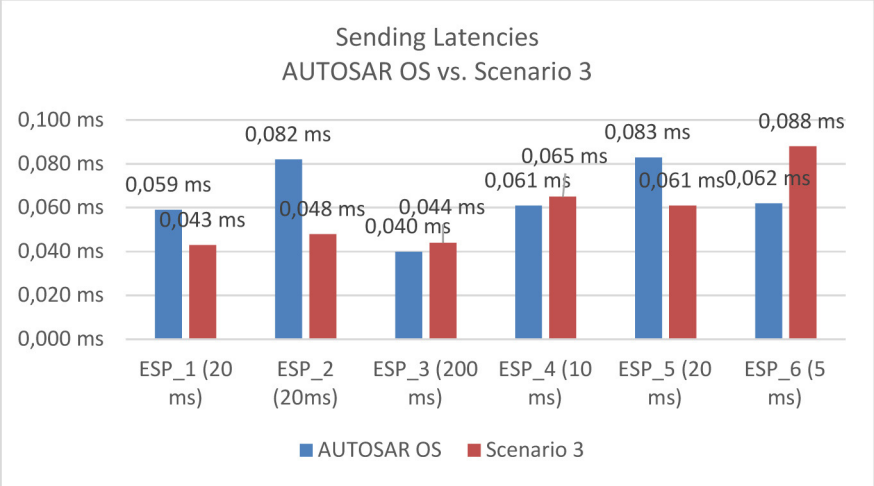

Figure 11. Sending latencies for AUTOSAR OS and Scenario 3

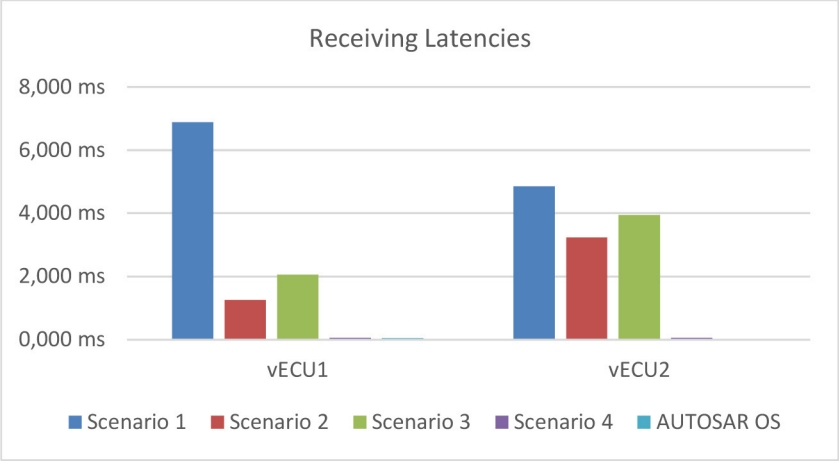

Figure 12. Receiving latencies

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

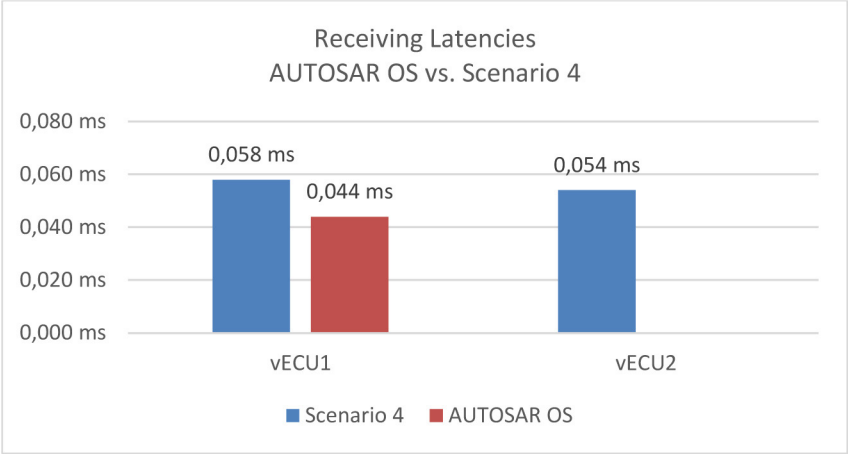

Figure 13. Receiving latencies for AUTOSAR OS and Scenario 4

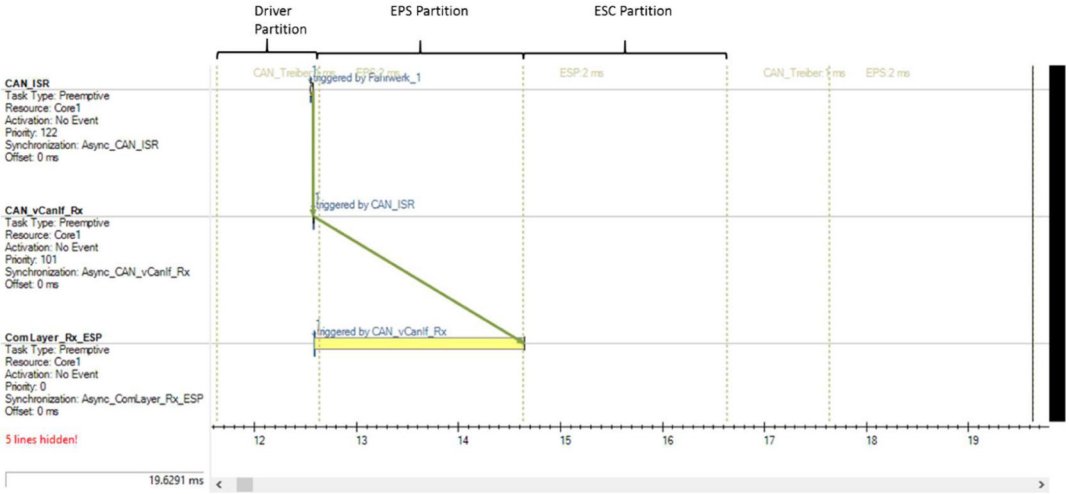

Figure 14. Gantt chart for best case latency in receiving scenario 1 and ESC as destination partition

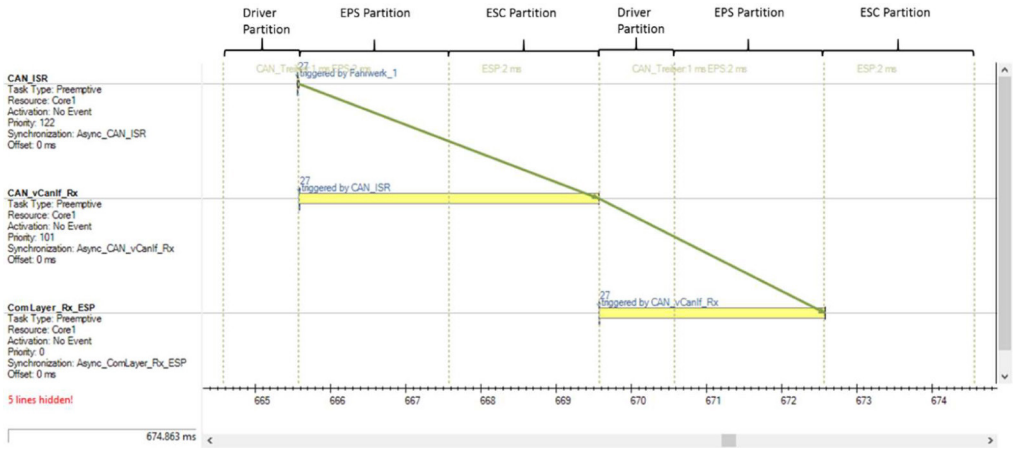

Figure 15. Gantt chart for worst case latency in receiving scenario 1 and ESC as destination partition

Downloaded from SAE International by Univ of California Berkeley, Monday, July 30, 2018

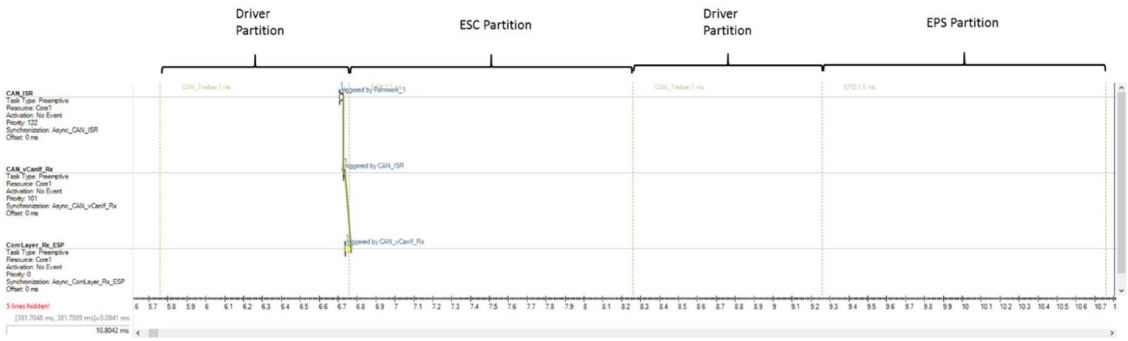

Figure 16. Gantt chart for best case latency in receiving scenario 2 and ESC as destination partition

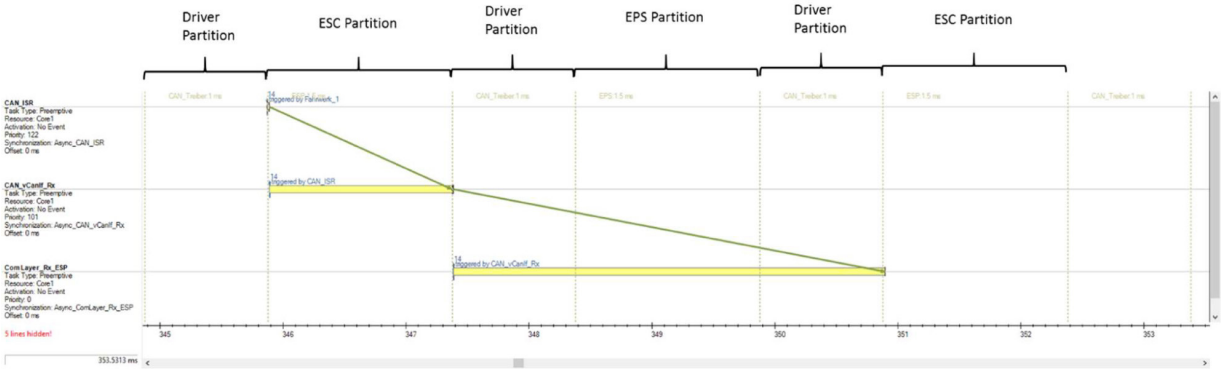

Figure 17. Gantt chart for worst case latency in receiving scenario 2 and ESC as destination partition

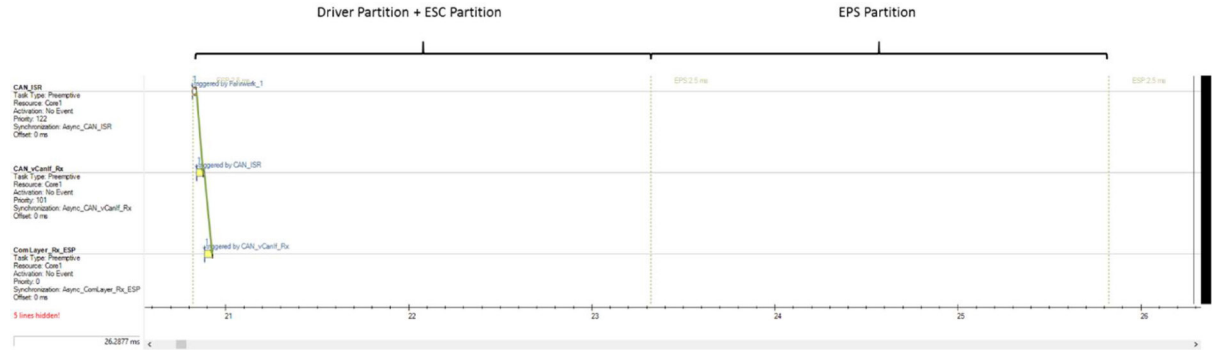

Figure 18. Gantt chart for best case latency in receiving scenario 3 and ESC as destination partition

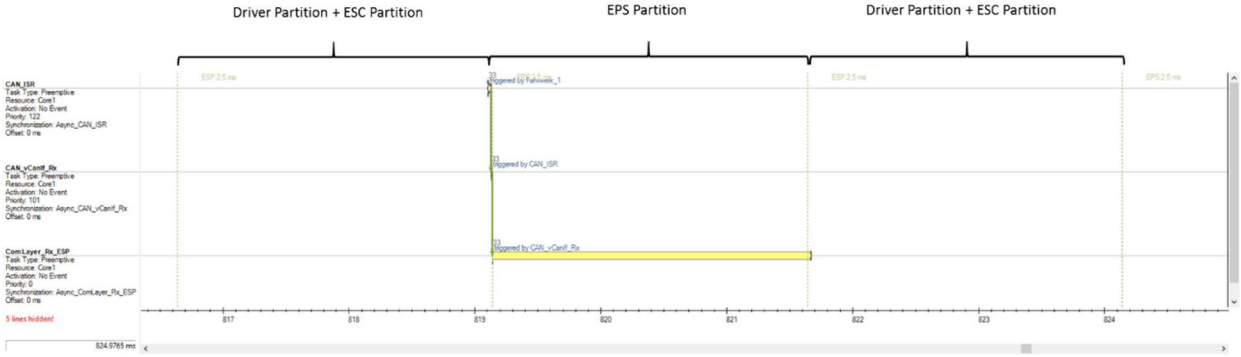

Figure 19. Gantt chart for worst case latency in receiving scenario 3 and ESC as destination partition

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of SAE International.

Positions and opinions advanced in this paper are those of the author(s) and not necessarily those of SAE International. The author is solely responsible for the content of the paper.